

UNIT III Basics of Python and Scikit Image

Python Basics: Variables, Data Types:

What Are Variables in Python?

DEFINITION

A variable in Python is a symbolic name that represents a value stored in memory. Variables are used to store data, which can then be manipulated or retrieved.

How to Declare a Variable

In Python, declaring a variable is as simple as assigning a value to it using the = operator. There is no need to explicitly declare the type of the variable, as Python is dynamically typed.

Example: program

Declaring variables

```
name = "Alice" # String
```

```
age = 25      # Integer
```

```
height = 5.7  # Float
```

Printing variables

```
print(name) # Output: Alice
```

```
print(age)  # Output: 25
```

```
print(height) # Output: 5.7
```

Rules for Naming Variables

1. Variable names must start with a letter or an underscore (_).
2. They can only contain letters, numbers, and underscores.
3. Variable names are case-sensitive (age and Age are different variables).
4. Reserved keywords (like if, for, print, etc.) cannot be used as variable names.

Data Types in Python

Python offers a variety of built-in data types to handle different kinds of data. Let's explore the primary data types and their uses.

1. Numeric Data Types

Numeric data types are used to store numbers and include:

- **Integer (int):** Whole numbers (e.g., 10, -5).
- **Float (float):** Decimal numbers (e.g., 3.14, -0.99).
- **Complex (complex):** Numbers with real and imaginary parts (e.g., 3+5j).

Example:

```
# Numeric types
```

```
a = 10    # Integer
```

```
b = 3.14  # Float
```

```
c = 2 + 3j # Complex
```

```
print(type(a)) # Output: <class 'int'>
```

```
print(type(b)) # Output: <class 'float'>
```

```
print(type(c)) # Output: <class 'complex'>
```

2. String (str)

Strings are sequences of characters enclosed in either single or double quotes.

Example:

```
# String examples
```

```
greeting = "Hello, World!"
```

```
char = 'A'
```

```
# String operations
```

```
print(greeting.upper()) # Output: HELLO, WORLD!
```

```
print(greeting.lower())    # Output: hello, world!
```

```
print(greeting[0:5])      # Output: Hello
```

3. Boolean (bool)

Booleans represent one of two values: True or False.

Example:

```
# Boolean examples
```

```
is_python_easy = True
```

```
is_maths_hard = False
```

```
print(is_python_easy)    # Output: True
```

```
print(type(is_python_easy)) # Output: <class 'bool'>
```

4. Sequence Types

Python includes three main sequence types:

- **List: Ordered, mutable collection.**
- **Tuple: Ordered, immutable collection.**
- **Range: Sequence of numbers.**

List Example:

```
fruits = ["apple", "banana", "cherry"]
```

```
print(fruits[0])    # Output: apple
```

```
fruits.append("orange") # Add an element
```

```
print(fruits)      # Output: ['apple', 'banana', 'cherry', 'orange']
```

Tuple Example:

```
coordinates = (10, 20, 30)
```

```
print(coordinates[1]) # Output: 20
```

5. Dictionary (dict)

Dictionaries store key-value pairs.

Example:

```
student = {"name": "Alice", "age": 25, "grade": "A"}  
print(student["name"]) # Output: Alice
```

Data Structures:

Data Structures are a way of organizing data so that it can be accessed more efficiently depending upon the situation. Data Structures are fundamentals of any programming language around which a program is built. Python helps to learn the fundamental of these data structures in a simpler way as compared to other programming languages.

[Python Tuple](#) is a collection of Python objects much like a list but Tuples are [immutable](#) in nature i.e. the elements in the tuple cannot be added or removed once created. Just like a List, a Tuple can also contain elements of various types.

In Python, tuples are created by placing a sequence of values separated by ‘comma’ with or without the use of parentheses for grouping of the data sequence.

Note: Tuples can also be created with a single element, but it is a bit tricky. Having one element in the parentheses is not sufficient, there must be a trailing ‘comma’ to make it a tuple.

Python Tuple Operations

In Python, Tuple is a sequence. Hence, we can concatenate two tuples with + operator and concatenate multiple copies of a tuple with "*" operator. The membership operators "in" and "not in" work with tuple object.

Python Expression	Results	Description
(1, 2, 3) + (4, 5, 6)	(1, 2, 3, 4, 5, 6)	Concatenation
('Hi!') * 4	('Hi!', 'Hi!', 'Hi!', 'Hi!')	Repetition
3 in (1, 2, 3)	True	Membership

Even if there is only one object in a tuple, you must give a comma after it. Otherwise, it is treated as a string.

Control Flow Statements

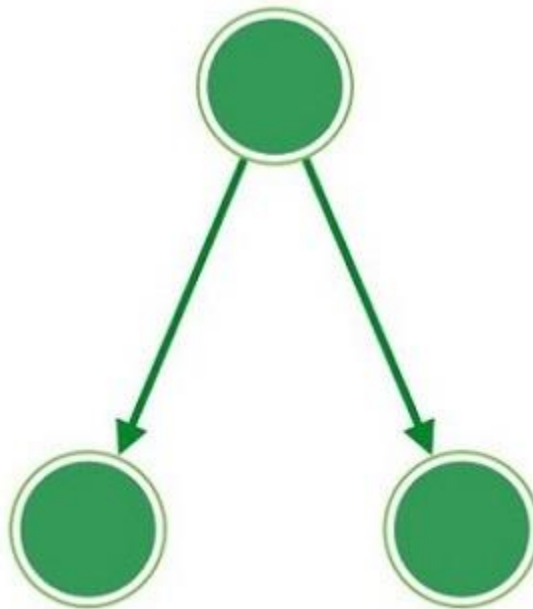
Python program control flow is regulated by various types of [conditional statements](#), [loops](#), and [function](#) calls. By default, the instructions in a computer program are executed in a sequential manner, from top to bottom, or from start to end. However, such sequentially executing programs can perform only simplistic tasks. We would like the program to have a decision-making ability, so that it performs different steps depending on different conditions.

Most programming languages including [Python](#) provide functionality to control the flow of execution of instructions. Normally, there are two type of control flow statements in any programming language and Python also supports them.

Decision Making Statements

Decision making statements are used in the Python programs to make them able to decide which of the alternative group of instructions to be executed, depending on value of a certain Boolean expression.

The following diagram illustrates how decision-making statements work –



The if Statements

Python provides [if..elif..else](#) control statements as a part of decision marking. It consists of three different blocks, which are **if** block, **elif** (short of else if) block and **else** block.

Example

Following is a simple example which makes use of if..elif..else. You can try to run this program using different marks and verify the result.

Open Compiler

```
marks = 80
```

```
result = ""
```

```
if marks < 30:
```

```
    result = "Failed"
```

```
elif marks > 75:
```

```
    result = "Passed with distinction"
```

```
else:
```

```
    result = "Passed"
```

```
print(result)
```

This will produce following result:

Passed with distinction

The match Statement

Python supports [Match-Case](#) statement, which can also be used as a part of decision making. If a pattern matches the expression, the code under that case will execute.

Example

Following is a simple example which makes use of match statement.

Open Compiler

```
def checkVowel(n):
```

```
    match n:
```

```
        case 'a': return "Vowel alphabet"
```

```
        case 'e': return "Vowel alphabet"
```

```
        case 'i': return "Vowel alphabet"
```

```
        case 'o': return "Vowel alphabet"
```

```
        case 'u': return "Vowel alphabet"
```

```
case _: return "Simple alphabet"
print (checkVowel('a'))
print (checkVowel('m'))
print (checkVowel('o'))
```

This will produce following result:

Vowel alphabet

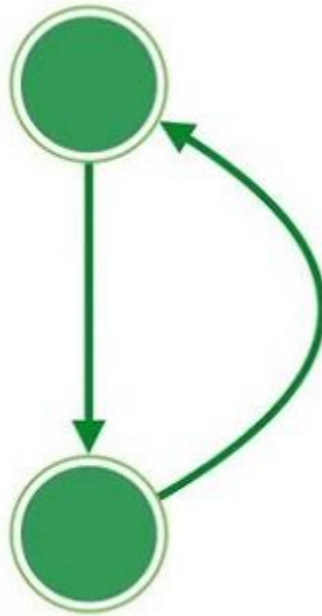
Simple alphabet

Vowel alphabet

Loops or Iteration Statements

*Most of the processes require a group of instructions to be repeatedly executed. In programming terminology, it is called a **loop**. Instead of the next step, if the flow is redirected towards any earlier step, it constitutes a loop.*

The following diagram illustrates how the looping works –



If the control goes back unconditionally, it forms an infinite loop which is not desired as the rest of the code would never get executed.

In a conditional loop, the repeated iteration of block of statements goes on till a certain condition is met. Python supports a number of loops like for loop, while loop which we will study in next chapters.

The for Loop

The [for loop](#) iterates over the items of any sequence, such as a list, tuple or a string .

Example

Following is an example which makes use of For Loop to iterate through an array in Python:

Open Compiler

```
words = ["one", "two", "three"]
```

```
for x in words:
```

```
    print(x)
```

This will produce following result:

one

two

three

The while Loop

The [while loop](#) repeatedly executes a target statement as long as a given boolean expression is true.

Example

Following is an example which makes use of While Loop to print first 5 numbers in Python:

Open Compiler

```
i = 1
```

```
while i < 6:
```

```
    print(i)
```

```
    i += 1
```

This will produce following result:

1

2

3

4

Jump Statements

The jump statements are used to jump on a specific statement by breaking the current flow of the program. In Python, there are two jump statements [break](#) and [continue](#).

The break Statement

It terminates the current loop and resumes execution at the next statement.

Example

The following example demonstrates the use of break statement –

Open Compiler

```
x = 0
```

```
while x < 10:
```

```
    print("x:", x)
```

```
    if x == 5:
```

```
        print("Breaking...")
```

```
        break
```

```
    x += 1
```

```
print("End")
```

This will produce following result:

```
x: 0
```

```
x: 1
```

```
x: 2
```

```
x: 3
```

```
x: 4
```

```
x: 5
```

```
Breaking...
```

End

The continue Statement

It skips the execution of the program block and returns the control to the beginning of the current loop to start the next iteration.

Example

The following example demonstrates the use of continue statement –

Open Compiler

for letter in "Python":

continue when letter is 'h'

if letter == "h":

continue

print("Current Letter :", letter)

This will produce following result:

Current Letter : P

Current Letter : y

Current Letter : t

Current Letter : o

Current Letter : n

Conditional Statements

Scikit Image:

Uploading and Viewing an Image

Pre-processing data refers to cleaning of data, removing invalid data, noise, replacing data with relevant values and so on. This doesn't always mean text data; it could also be images or video processing as well.

Data pre-processing basically refers to the task of gathering all the data (which is collected from various resources or a single resource) into a common format or into uniform datasets (depending on the type of data). Since real-world data is never ideal, there is a possibility that the data would have missing cells, errors, outliers, discrepancies in columns, and much more.

Sometimes, images may not be correctly aligned, or may not be clear or may have a very large size. The goal of pre-processing is to remove these discrepancies and errors.

Let us take the example of uploading an image and viewing it on console using scikit-learn library —

Example

```
from skimage import io
path = "path to puppy.PNG"
img = io.imread(path)
print("Image being read")
io.imshow(img)
print("Image printed on console")
```

Output

Image being read
Image printed on console



Explanation

- The required libraries are imported.
- The path where the image is stored is defined.
- The 'imread' function is used to visit the path and read the image.
- After the image is read, the pixel values are stored in the form of an array.
- This array is nothing but a Numpy array.
- The image is read and converted into an array.
- The 'imshow' function is used to display the image on the console.
- The data is displayed on the console.

Getting Image Resolution

1. Introduction to Image Resolution

Image Resolution refers to the amount of detail an image holds. It is typically described in **pixels**, denoting **width** × **height**. For example, a 1920×1080 image has:

- 1920 pixels horizontally (width)
- 1080 pixels vertically (height)

Higher resolution means more image detail.

2. About Scikit-Image (skimage)

Scikit-Image is a powerful Python library built on top of NumPy, used for:

- Image processing
- Image analysis
- Filtering and transformation
- Feature extraction and more

It supports many image formats and can handle grayscale, RGB, and multi-dimensional images.

3. Loading an Image

To work with images, we first load them using `skimage.io.imread()`:

python

CopyEdit

```
from skimage import io
```

```
# Load image
```

```
image = io.imread('example.jpg')
```

This function reads the image into a **NumPy array**.

4. Accessing Image Resolution

Once the image is loaded, its resolution (dimensions) can be accessed using `.shape`:

python

CopyEdit

```
height, width = image.shape[:2]
```

```
print(f'Resolution: {width} x {height}')
```

 **Explanation:**

- `image.shape` returns a tuple:
 - For **RGB** images: (height, width, channels)

- For **Grayscale** images: (height, width)
- `image.shape[:2]` extracts just height and width.

Example:

If `image.shape` = (720, 1280, 3)
 Then `width` = 1280, `height` = 720
 So resolution = **1280 x 720 pixels**

5. Visual Confirmation

You can also display the image and confirm resolution visually:

python

`import matplotlib.pyplot as plt`

`plt.imshow(image)`

`plt.title(f'Resolution: {width} x {height}')`

`plt.axis('off')`

`plt.show()`

6. Differentiating Pixel Resolution vs DPI

- **Pixel Resolution:** Total pixels in width and height (using `.shape`)
- **DPI (Dots Per Inch):** Used in printing, not directly handled by skimage

To access DPI (if needed), you can use **Pillow (PIL)**:

python

`from PIL import Image`

`img = Image.open('example.jpg')`

`print(img.info.get('dpi', 'DPI not found'))`

7. Applications of Getting Image Resolution

- Verifying image quality before processing
- Resizing images for web or printing
- Detecting large or small images in datasets

- Maintaining aspect ratio while resizing

8. Comparison with Other Libraries

Library	Method to Get Resolution	Additional Info Available
scikit-image	image.shape	Pixel dimensions only
Pillow	img.size and img.info['dpi']	Pixel + DPI info
OpenCV	image.shape	Similar to skimage

9. Sample Output

Resolution: 1280 x 720

This means the image has 1280 pixels in width and 720 in height.

10. Conclusion

Using **Scikit-Image**, getting image resolution is simple and effective for many computer vision tasks. While it does not provide DPI, it excels at analyzing pixel dimensions, which are essential in digital image processing.

Looking at Pixel Values,

When working with images in Python, **Scikit-Image** (skimage) is a popular library for image processing tasks. Images are essentially multidimensional arrays (usually NumPy arrays), where each element corresponds to a pixel's value (or values for multi-channel images like RGB).

Loading an Image

First, you need to load an image into a NumPy array:

```
from skimage import io
```

```
image = io.imread('path_to_image.jpg')
```

```
print(type(image)) # <class 'numpy.ndarray'>
```

```
print(image.shape) # e.g. (height, width, channels) for RGB
```

Understanding Pixel Values

- For **grayscale images**, the array is 2D: [height, width]. Each pixel holds a single intensity value, often ranging from 0 to 255 (uint8) or [0, 1] (float).
- For **color images** like RGB, the array is 3D: [height, width, 3]. Each pixel holds 3 values for Red, Green, and Blue channels.

Accessing Pixel Values

You can access the pixel values by using array indexing:

```
# Access pixel at row 50, column 100
```

```
pixel_value = image[50, 100]
```

```
print(pixel_value)
```

- If the image is grayscale, pixel_value might be something like: 123
- If the image is RGB, pixel_value will be a vector like: [255, 128, 64]

Modifying Pixel Values

You can also change pixel values directly:

```
# Set pixel at row 50, column 100 to red (for RGB images)
```

```
image[50, 100] = [255, 0, 0]
```

Viewing Pixel Values with Visualization and Print

Sometimes you want to sample a small region or print multiple pixels:

```
# Print pixel values in a 5x5 region at top-left corner
```

```
print(image[0:5, 0:5])
```

You can also visualize the image with pixel grids using Matplotlib:

```
import matplotlib.pyplot as plt
```

```
plt.imshow(image)
```

```
plt.title('Image')
```

```
plt.show()
```

Important Tips

- The data type of the image array (image.dtype) often determines the pixel value range:
 - uint8: 0-255

- float: 0.0-1.0
- Use `skimage.img_as_float` or `skimage.util.img_as_ubyte` to convert image data types if needed.

Example: Complete Code

```
from skimage import io, img_as_float

# Load image as float type (values between 0 and 1)
image = img_as_float(io.imread('example.jpg'))

print(f'Image shape: {image.shape}')
print(f'Data type: {image.dtype}')

# Access pixel value at (10, 20)
pixel = image[10, 20]
print(f'Pixel value at (10,20): {pixel}')

# Modify pixel to white (for float, white is [1,1,1])
image[10, 20] = [1, 1, 1]

# Show image
import matplotlib.pyplot as plt
plt.imshow(image)
plt.show()
```

Summary

- **Images are NumPy arrays** in scikit-image.
- Access pixel values by indexing, e.g., `image[row, col]`.
- Values depend on image type and format.

- You can print, modify, and visualize pixel data easily.

This approach gives you full control over pixel-level manipulation, useful in many image processing tasks such as filtering, segmentation, and annotation.

Converting Color Space

Color space conversion is a fundamental process in digital image processing that involves transforming an image from one color representation to another. Scikit-Image, a Python library, offers built-in functions to easily switch between different color models such as RGB, HSV, LAB, and YCbCr. This is essential for various applications like segmentation, filtering, compression, and enhancement.

What is a Color Space?

A **color space** defines how colors are represented using numerical values. Each color space arranges colors differently to highlight particular features such as hue, brightness, or chrominance.

Common Color Spaces:

- **RGB** – Red, Green, Blue (default format for most images)
- **HSV** – Hue, Saturation, Value (useful for color-based processing)
- **LAB** – Lightness, A (green–red), B (blue–yellow), perceptually uniform
- **YCbCr (YUV)** – Used in video compression (Y = luminance, Cb/Cr = chroma)

Why Convert Color Spaces?

Converting color spaces enables:

- Better object segmentation (e.g., using hue in HSV)
- Enhanced filtering or thresholding based on lightness
- More accurate color comparisons (in LAB)
- Efficient storage and compression (in YCbCr)

Using Scikit-Image for Conversion

To use Scikit-Image for color space conversion:

```
from skimage import io, color
```

```
image = io.imread('image.jpg')
```

Examples:

1. RGB to Grayscale

```
gray = color.rgb2gray(image)
```

2. RGB to HSV

```
hsv = color.rgb2hsv(image)
```

3. RGB to LAB

```
lab = color.rgb2lab(image)
```

4. RGB to YCbCr

```
ycbcr = color.rgb2ycbcr(image)
```

Reverse Conversion:

```
rgb = color.hsv2rgb(hsv)
```

```
rgb = color.lab2rgb(lab)
```

Applications of Color Space Conversion

Application	Best Color Space
Skin/face detection	HSV
Image segmentation	LAB
Video compression	YCbCr
Brightness filtering	Grayscale

Conclusion

Color space conversion is a key technique in image processing. Scikit-Image simplifies this process with easy-to-use functions. Choosing the appropriate color space—such as HSV for color segmentation or LAB for visual perception—improves the accuracy and efficiency of many image processing tasks.

